

```
_ATTR(name, permission, show_ptr, store_ptr)
```

...where, *name* is the name with which the attribute is to be created, *permission* is file permission where 0666 is read and write permission, *show_ptr* is the function pointer called when the file is read, and *store_ptr* is the function pointer called when the file is written.

The *example_attr* defined above is then grouped in the *struct attribute* group as follows:

```
struct attribute *example_attr[] = {
    &example_attr.attr,
    NULL,
};
```



Note: In the *struct attribute*, multiple attributes can be grouped together and then be created in one single API call. Also, NULL termination is compulsory for the group.

The above attributes are then given to the attribute group as follows:

```
struct attribute_group example_attr_group = {
    .attrs = example_attr,
};
```

To create attributes, the following *sysfs* API is used:

```
int sysfs_create_group(struct kobject *kobj, const struct
attribute_group *grp);
```

...where,

kobj: This is the pointer to the kobject of the directory in which the file is to be created.

grp: This is the group of attributes to be created, which will be created as files in *sysfs*.

To create the *example_info* file in our */sys/kernel/example_dev* directory, the following *sysfs* API is called:

```
sysfs_create_group(example_dev_kobj,&example_attr_group);
```

After this function, a file named *example_info* will be created in the */sys/kernel/example_dev* directory.

Now, whenever the file is read, the *example_show* function will be called and whenever the file is written, the *example_store* function will be called, which needs to be defined. The function prototype for the show and store function is as follows:

```
ssize_t (*show)(struct kobject * kobj, struct attribute *
attr, char * buff);
```

...where,

kobj: This is the pointer to the kobject of the directory

which was passed when the file was created.

attr: This is the attribute pointer of the file which was passed when file was created.

buff: This is the buffer in which the output should be written, which will be displayed as the result in user space. The value written should not exceed the size defined by the *PAGE_SIZE* macro.

```
ssize_t (*store)(struct kobject * kobj, struct attribute *
attr, const char * buff, size_t size);
```

...where, *kobject* and *attr* are as described above.

buff: This buffer contains the value that was written to the file from user space.

size: This is the size of the data written to the file through user space.

So our *example_show* function can be defined as follows:

```
ssize_t example_show(struct kobject *kobj, struct kobj_
attribute *attr, char *buf)
{
    return sprintf(buf, "example show");
}
//Here any value of variable can be written and exposed to
user space through this interface
also return value is the size of data written to the
buffer*/
```

...and an example store can be written as follows:

```
static ssize_t example_store(struct kobject *kobj, struct
kobj_attribute *attr, const char *buf, size_t count)
{
    //here the value from the buff can be read and //respective
action can be taken by module

    return count;

}
//return the number of bytes read, if all the //bytes are
read the count parameter which //was received by store
routine
```

Driver sysfs interface

Drivers are modules written to interface user space with hardware devices. Hardware devices have information, which the user needs to know during their operation. So drivers generally create *sysfs* files for providing their information to user space.

Drivers don't explicitly create directories in *sysfs*.

Whenever drivers register with their sub-system, directories are internally created for the driver. Each driver has access to the *struct device* pointer for the device for which the driver is being written. This device pointer is used to create files in the driver interface.